

Lucrare practică nr. 4

Tema: Aplicații ale rețelei neuronale recurente (RNN)

Scopul lucrării: Implementarea unei RNN în Python

Obiectivele lucrării:

1. Preprocesarea limbajului natural. Folosirea vocabularelor română și engleză.
2. Crearea modelului, antrenarea și optimizarea rețelei.
3. Compararea rezultatelor cu alte modele: LSTM și GRU.

Numărul de ore necesare pentru realizare – 4 ore academice.

8.1 Noțiuni de bază:

Rețeaua neuronală recurentă (RNN) este un tip de rețea neuronală în care ieșirea de la pasul anterior este alimentată ca intrare în pasul curent [21].

Caracteristica principală și cea mai importantă a RNN este *starea* sa ascunsă, care reține unele informații despre o secvență. Starea este denumită și *starea memoriei*, deoarece își amintește intrarea anterioară în rețea.

RNN are multiple utilizări:

- în domeniul financiar: RNN poate fi de ajutor în prezicerea prețurilor acțiunilor sau a semnelor direcției pieței de valori (adică pozitiv sau negativ);
- la dirijarea mașinii autonome, poate evita un accident de mașină anticipând traiectoria vehiculului;
- în analiza textului (revizuiți materialul despre procesare texte din îndrumarul pentru cursul Tehnici de programare în Python [19]), analiza sentimentelor din text [22], traducerea automată;
- ș.a.

8.2 Sarcina individuală:

1. Alegeți o problemă care să necesite implementarea unei RNN.
2. Creați modelul.
3. Calculați pierderile, optimizați, faceți predicții/testați (în timp real) cu noi valori.
4. Comparați modelul creat cu alte modele RNN, de ex. LSTM și GRU.
5. Scrieți concluzii

Notă: Puteți crea un model *Secvențial* cu ***SimpleRNNCell***.

```
model = Sequential()
model.add(Input(shape=(1,), dtype="string"))
max_tokens = 1000
max_len = 100
vectorize_layer = TextVectorization(
    max_tokens=max_tokens,
    output_mode="int",
    output_sequence_length=max_len,
)
train_texts = train_data.map(lambda text, label: text)
```

```

vectorize_layer.adapt(train_texts)

model.add(vectorize_layer)
model.add(Embedding(max_tokens + 1, 128))

rnn = RNN(SimpleRNNCell(64) ,
return_sequences=False, return_state=False)
model.add(rnn)
model.add(Dense(64, activation="relu"))
model.add(Dense(1, activation="sigmoid"))

```

Puteți modela o rețea neuronală de tip **LSTM** (LSTM, acronim pentru Long Short Term Memory) ori **Bidirectional(LSTM)** pentru prezicerea seriilor temporale.

```

model = Sequential()
...
model.add(LSTM(64))
model.add(Dense(64, activation="relu"))
model.add(Dense(1, activation="sigmoid"))

```

Puteți modela o rețea neuronală de tip **GRU** (Gated Recurrent Unit) ori (**Bidirectional(GRU)**).

```

model = Sequential()
...
model.add(GRU(64))
model.add(Dense(64, activation="relu"))
model.add(Dense(1, activation="sigmoid"))

```

Pentru rețele de tip GRU și LSTM puteți testa ambele variante : cu și fără **Bidirectional** și comparați rezultatele obținute.

8.3 Evaluare și livrabile

Studentii vor demonstra cunoștințele și abilitățile dobândite ca urmare a realizării lucrării practice. Vor explica pașii creării unei rețele neuronale recurente, vor face o comparație cu alte modele RNN, vor da exemple de aplicare a RNN.

Studentii vor prezenta un fișier cu extensia .py și un raport .doc care conține: tema, obiectivele, sarcina, codul sursă și capturi de ecran cu rezultatele obținute la rularea codului și o concluzie despre ce au reușit să asimileze efectuând sarcinile din lucrare.

Modelul de raport poate fi consultat în Anexa 1.

Întrebări de autoevaluare:

1. Care sunt pașii de creare a unei rețele neuronale recurente (RNN)?
2. Care sunt librăriile destinate procesării limbajului natural (NLP) în Python? Care sunt avantajele fiecărei librării?
3. Care sunt pașii de procesare a textului pentru a fi transformat în date de intrare în straturile Dense?
4. Cum alegem numărul de straturi rnn? Dar numărul de straturi Dense într-o rețea RNN?

5. De ce este necesară transformarea textului cu `tf.keras.layers.TextVectorization`?
6. Ce este `tf.keras.layers.Embedding`?
7. Ce funcții de activare folosim în straturile interne, dar în stratul de ieșire?
8. Ce funcție este folosită la calcularea funcției `loss`?
9. Care este diferența între rețeaua neuronală de tip LSTM (Long Short Term Memory) și Bidirectional LSTM? Dați exemple concrete de aplicare.
10. Care este diferența între rețeaua neuronală de tip GRU (Gated Recurrent Unit) și Bidirectional GRU? Dați exemple concrete de aplicare.